Anomalous-curve follow-up: QTT structure of the transfer matrix

This is a follow-up to holant step (b) (docs/reports/holant-step-b-qtt-20261104). In that run the single curve whose x-labelling scored below its random-relabelling null was p = 61, a = 21, b = 57. It has #E = p, so it is **anomalous**, the class where Smart's attack makes the DLP easy. This study tests whether that was a property of anomalous curves or chance.

Protocol protocol.json, ANOMALOUS-QTT-20261104, was frozen before data (sha256 82703e7f25ed…, commit ef35b300). The run script, the core (a byte copy of step (b)'s frozen qtt_rank.py) and the analysis script are all pinned by hash.

Verdict (frozen rule): NULL, with valid controls. Anomalous curves show no QTT structure beyond ordinary prime-order curves at p ≤ 127. The p = 61 observation was chance. This is a structure statistic, not a cost claim.

Design

- **Curves.** All 20 primes 37 ≤ p ≤ 127, where QTT ranks are not saturated. For each prime, up to 4 anomalous isomorphism classes ((a u^4, b u^6) orbits) are drawn by a protocol-hash-seeded stream, together with an equal number of ordinary prime-order classes. That gives 63 + 63 curves. The motivating curve's class is **excluded** from the test and reported separately.
- **Per curve:**
  - X, M in natural x-labelling;
  - LOG, the discrete-log orbit labelling (positive control);
  - 99 random relabellings (the null). Draw 0 scored against draws 1–98 is the A/A check.

  FB is the $\lfloor\sqrt{p}\rfloor$ smallest x-coordinates on E.
- **Primary metric.** u_X is the mid-rank percentile of X's interleaved QTT parameter count (Σ $\log_2$ rank) within that curve's own null, so each curve is compared with itself. The secondary metric is the same with r_max.
- **Test.** Mean u_X(ordinary) − mean u_X(anomalous), with a one-sided label permutation stratified by prime (9999 resamples).
- **Rule.**
  - EFFECT iff p < 0.01 and mean u_X(anomalous) < 0.25.
  - NULL iff p ≥ 0.05.
  - Validity requires mean u_LOG < 0.05 in each group and A/A in [0.35, 0.65].

Results

metric	verdict	p (perm.)	mean u_X anomalous	mean u_X ordinary	u_LOG (anom, ord)	A/A
Σ $\log_2$ rank (primary)	**NULL**	0.831	0.534	0.483	0.00, 0.00	0.506
r_max (secondary)	NULL	0.620	0.493	0.478	0.00, 0.00	0.499

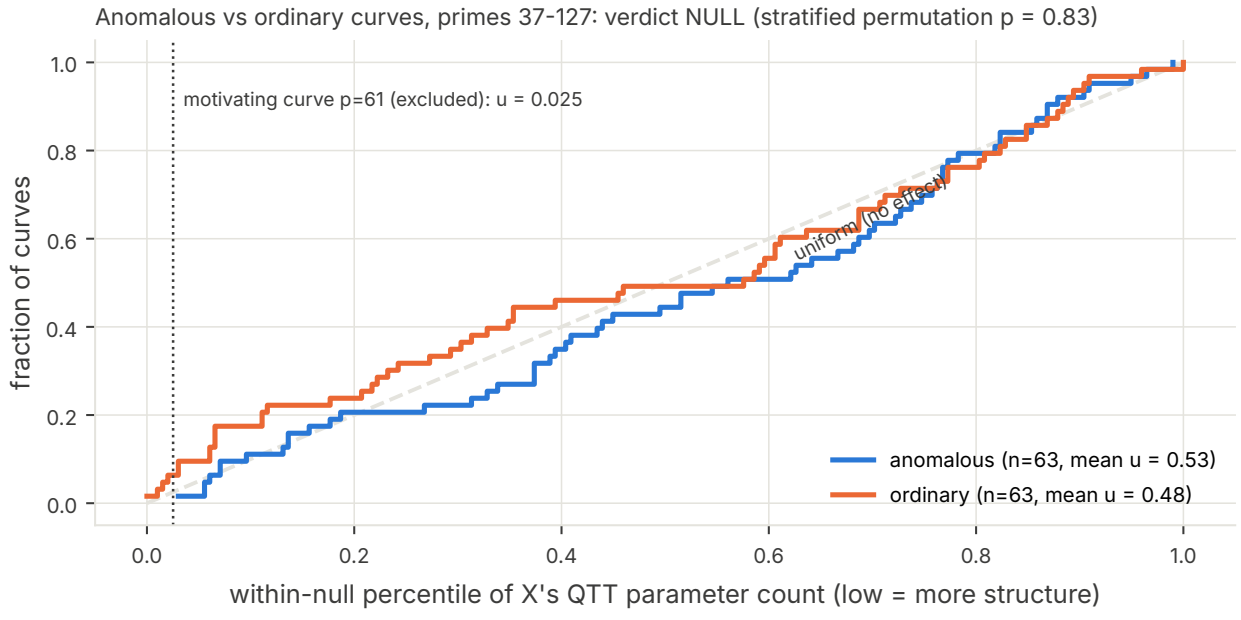

Figure 1: ECDF of within-null percentiles

- The anomalous curves sit slightly **above** the ordinary ones, the opposite of the hypothesised direction. Both ECDFs follow the uniform line.
- 1 of 63 anomalous curves (1.6%) fell below the 5th percentile, against 6 of 63 ordinary curves (9.5%).
- The motivating curve, rerun under this protocol's seeds, again sits low ($u_X = 0.025$). At $u \le 0.025$ there are 4 ordinary curves and 0 anomalous ones among the 126, which is 4 against 3.2 expected under the null. So it is a tail draw, and the tail is not anomalous-specific. (Wording corrected after independent validation.)
- Both controls behave as designed. LOG is below every null draw on every curve ($u = 0$), and the A/A mean is 0.51.

Consequence

The pointer recorded in the holant step-(b) report and in the lead row is closed: at $p \le 127$, anomalous curves show no anomaly-specific digit structure in the x-labelled decomposition network. Smart's attack exploits the p-adic formal group, not anything this network or its digit labelling sees.

Deviations and notes

- **Export checker mismatch.** The export (export.json.xz) fails check-export for the same encoding mismatch as step (b) (amendment A1 there). The null draws are not encoded as replicates, the script's seed formula is not the checker's, and the motivating curve's arm is reported outside the export's arm list. The counts in the export match the rows.
- **Compression.** Rows and export are stored as .xz to stay under the 64 KiB in-Git threshold. Uncompressed hashes are in data.sha256.
- **No S3 checkpoint.** The container has no upload token. With the owner's approval (session 2026-11-04), the pushed branch is the resume record. The run took about 2 minutes, with exact counts and no clock.

Reproduce

```
xz -dc rows.jsonl.xz > rows.jsonl     # or rerun:
python3 anomalous_qtt.py --protocol-sha 82703e7f25eda9775ec61e7d25dcbc53aeced794f26f740707c5a046a4ec3f84 --out rows.jsonl --include-excluded
python3 analyze_anomalous.py rows.jsonl verdict.json log2_params
python3 analyze_anomalous.py rows.jsonl verdict_rmax.json r_max
python3 figures.py; ./build_pdf.sh
```